# GHOSTKEY: SECURE INTELLIGENT CLOUD STORAGE WITH CRYPTOGRAPHIC PROTECTION

**A PROJECT REPORT**

***Submitted by***


| | |
|---|---|
| **KISHORE M** | **811722104079** |
| **LOGESHWARAN P** | **811722104083** |
| **MADHAVAN P** | **811722104084** |


***in partial fulfillment of the requirements for the award degree of***

***Bachelor of Engineering***

**20CS8504 PROJECT WORK**


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY**

**(AUTONOMOUS)**

**SAMAYAPURAM – 621112**


**APRIL 2026**

# GHOSTKEY: SECURE INTELLIGENT CLOUD STORAGE WITH CRYPTOGRAPHIC PROTECTION

**A PROJECT REPORT**

***Submitted by***


| | |
|---|---|
| **KISHORE M** | **811722104079** |
| **LOGESHWARAN P** | **811722104083** |
| **MADHAVAN P** | **811722104084** |


***in partial fulfillment of the requirements for the award degree of***

***Bachelor of Engineering***

**20CS8504 PROJECT WORK**


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY**

**(AUTONOMOUS)**

**SAMAYAPURAM – 621112**


**APRIL 2026**

# K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

# (AUTONOMOUS)

# SAMAYAPURAM - 621112

## BONAFIDE CERTIFICATE

The work embodied in the present project report entitled "**GHOSTKEY: SECURE INTELLIGENT CLOUD STORAGE WITH CRYPTOGRAPHIC PROTECTION**" has been carried out by the students **KISHORE M, LOGESHWARAN P, MADHAVAN P**, The work reported here in is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for assessment.

Date of Viva Voce: ……

| | |
|---|---|
| **Mrs. V. KALPANA, M.E.,(Ph.D.,)** | **Mr. R. RAJAVARMAN, M.E.,(Ph.D.,)** |
| SUPERVISOR | HEAD OF THE DEPARTMENT |
| Assistant Professor | Assistant Professor (Sr. Grade) |
| Department of CSE | Department of CSE |
| K. Ramakrishnan College of Technology | K. Ramakrishnan College of Technology |
| Samayapuram -621 112 | Samayapuram -621 112 |

**INTERNAL EXAMINER** **EXTERNAL EXAMNIER**

# ABSTRACT

In the modern digital era, cloud computing has become a vital platform for storing and managing data efficiently. However, the growing reliance on cloud services has introduced critical security challenges such as unauthorized access, data breaches, and data integrity issues. Ensuring secure storage and controlled data access is therefore essential for protecting sensitive information in cloud environments. This project presents a secure intelligent cloud storage with cryptographic protection system that enhances data security through the integration of advanced cryptographic and intelligent techniques. The system utilizes AES encryption to secure data before storage, sha-256 hashing to ensure data integrity, and multi-factor authentication (MFA) with OTP verification to strengthen user authentication. Additionally, intelligent monitoring mechanisms are incorporated to track user activities and identify potential security threats. The proposed system enables users to securely upload, store, and access data while maintaining confidentiality, integrity, and authentication. It provides a reliable, efficient, and scalable solution for secure cloud storage, making it suitable for real-world applications.

**Keywords:** Cloud Security, Cryptography, AES Encryption, SHA-256, Multi-Factor Authentication, Secure Cloud Storage, Data Integrity

# ACKNOWLEDGEMENT

We thank our **Dr. N. Vasudevan**, Principal, for his valuable suggestions and support during the course of my research work.

We thank our **Mr. R. Rajavarman,** Head of the Department, Assistant Professor (Sr. Grade), Computer Science and Engineering, for his valuable suggestions and support during the course of my research work.

We wish to record my deep sense of gratitude and profound thanks to my Guide **Mrs. V. Kalpana,** Assistant Professor, Computer Science and Engineering for her keen interest, inspiring guidance, constant encouragement with my work during all stages, to bring this thesis into fruition.

We are extremely indebted to our project coordinator **Mrs. M. Mathumathi,** Assistant Professor, Computer Science and Engineering, for her valuable suggestions and support during the course of my research work.

We also thank the faculty and non-teaching staff members of the Computer Science and Engineering, K. Ramakrishnan College of Technology (Autonomous), Samayapuram, for their valuable support throughout the course of my research work. Finally, we thank our parents, friends and our well wishes for their kind support.

**SIGNATURE**

_____

_____

_____

# TABLE OF CONTENTS


| CHAPTER No. | TITLE | PAGE No. |
|---|---|---|
| | **ABSTRACT** | **iii** |
| | **LIST OF FIGURES** | **vii** |
| | **LIST OF SYMBOLS AND ABBREVIATIONS** | **viii** |
| **1** | **INTRODUCTION** | **1** |
| | 1.1 DESCRIPTION | 2 |
| | 1.2 OVERVIEW | 2 |
| | 1.3 OBJECTIVE | 3 |
| **2** | **LITERATURE SURVEY** | **5** |
| **3** | **EXISTING SYSTEM** | **15** |
| **4** | **PROBLEMS IDENTIFIED** | **18** |
| **5** | **PROPOSED SYSTEM** | **20** |
| **6** | **SYSTEM REQUIREMENTS** | **22** |
| | 6.1 HARDWARE REQUIREMENTS | 22 |
| | 6.2 SOFTWARE REQUIREMENTS | 22 |
| **7** | **SYSTEM IMPLEMENTATIONS** | **23** |
| | 7.1 LIST OF MODULES | 23 |
| | 7.1.1 User Authentication Module | 23 |
| | 7.1.2 OTP Generation and Verification Module | 24 |
| | 7.1.3 File Encryption and Upload Module | 24 |
| | 7.1.4 Secure Cloud Storage Module | 25 |
| | 7.1.5 File Sharing and Activity Monitoring Module | 25 |
| **8** | **SYSTEM TESTING** | **26** |
| | 8.1 UNIT TESTING | 26 |
| | 8.2 INTEGRATION TESTING | 27 |
| | 8.3 SYSTEM TESTING | 27 |
| | 8.4 PERFORMANCE TESTING | 28 |
| | 8.5 USER ACCEPTANCE TESTING (UAT) | 29 |

| | | |
|---|---|---|
| **9** | **RESULTS AND DISCUSSION** | **30** |
| **10** | **CONCLUSION AND FUTURE WORK** | **32** |
| | 10.1 CONCLUSION | 32 |
| | 10.2 FUTURE ENHANCEMENTS | 33 |
| | **APPENDIX A** | **35** |
| | **APPENDIX B** | **44** |
| | **REFERENCES** | **47** |

# LIST OF FIGURES


| FIGURE NO. | FIGURE NAME | PAGE NO. |
|---|---|---|
| 3.1 | EXISTING SYSTEM ARCHITECTURE | 16 |
| 5.1 | PROPOSED SYSTEM ARCHITECTURE | 21 |
| B.1 | USER REGISTRATION AND AUTHENTICATION | 44 |
| B.2 | DASHBOARD MANAGEMENT | 44 |
| B.3 | FILE ENCRYPTION AND DECRYPTION | 45 |
| B.4 | SECURE CLOUD STORAGE | 45 |
| B.5 | FILE SHARING AND ACTIVITY MONITORING | 46 |

# LIST OF ABBREVIATIONS

| | | |
|---|---|---|
| AI | – | Artificial Intelligence |
| AES | – | Advanced Encryption Standard |
| OTP | – | One-Time Password |
| TOTP | – | Time-Based One-Time Password |
| MFA | – | Multi-Factor Authentication |
| SHA-256 | – | Secure Hash Algorithm 256-bit |
| DB | – | Database |
| HTML | – | Hypertext Markup Language |
| CSS | – | Cascading Style Sheets |
| JS | – | JavaScript |
| Flask | – | Python Web Framework |
| SQL | – | Structured Query Language |

# CHAPTER 1

# INTRODUCTION

The use of cloud storage systems has increased significantly in recent years, as individuals and organizations depend on them to store and share important data. However, many existing cloud storage solutions lack strong security mechanisms, making them vulnerable to issues such as unauthorized access, data breaches, and data tampering. As a result, users often face risks related to loss of confidentiality and integrity of their data. These challenges highlight the need for a secure and reliable system that can protect sensitive information in cloud environments.

With advancements in security technologies, it is possible to design systems that ensure safe data storage and sharing. The Secure Intelligent Cloud Storage with Cryptographic Protection (GhostKey) is developed to address these security concerns by integrating encryption, authentication, and data integrity techniques. The system provides a secure platform where only authenticated users can upload, access, and share files, thereby reducing the risk of unauthorized access.

In this system, files are encrypted using AES encryption before being stored in the cloud, ensuring that the data remains confidential. To maintain data integrity, SHA-256 hashing is used to verify that files are not modified during storage or transmission. Additionally, the system incorporates Multi-Factor Authentication (MFA) using dynamic OTP verification, which adds an extra layer of security by validating user identity.

The main objective of this project is to provide a secure and efficient cloud file-sharing environment. It enables users to safely upload, store, and share files while ensuring data protection, integrity, and user privacy.

## 1.1 DESCRIPTION

The Secure Intelligent Cloud Storage with Cryptographic Protection, also known as GhostKey, is a web-based application designed to provide secure file storage and sharing services. The system allows users to register and authenticate themselves using OTP-based verification, ensuring that only authorized users can access the platform.

Once authenticated, users can upload files, which are automatically encrypted using AES encryption before being stored in the cloud. This ensures that even if unauthorized access occurs, the data remains protected. The system also generates a SHA-256 hash value for each file to maintain data integrity and detect any modifications.

Users can securely share files with other authorized users, and all activities are monitored and recorded. The system also provides features such as file decryption, access control, and activity tracking, ensuring complete security and transparency. The main goal of the system is to enhance data security in cloud environments while maintaining ease of use and efficiency.

## 1.2 OVERVIEW

The Secure Intelligent Cloud Storage with Cryptographic Protection system is designed to provide a secure platform for cloud-based data storage and sharing. It integrates multiple domains such as cloud security, cryptography, authentication, artificial intelligence, and web technologies to ensure complete data protection. Cloud security plays an important role in safeguarding data by implementing measures such as encryption, authentication, and access control to prevent unauthorized access and data breaches. Cryptography is used to secure digital data through encryption and decryption techniques, where AES encryption protects files and SHA-256

hashing ensures data integrity during storage and transmission. Authentication systems are implemented to verify user identity, and the use of Multi-Factor Authentication (MFA) with OTP verification strengthens access control. Artificial Intelligence is incorporated to enhance system security by monitoring user activities and detecting unusual behavior, enabling proactive threat detection. The system is developed as a web-based application using technologies such as HTML, CSS, JavaScript, and Python with the Flask framework, providing an interactive and user-friendly interface. A database management system is used to store user data, encrypted files, hash values, and system logs, ensuring efficient data storage, retrieval, and activity monitoring. Overall, the system offers a secure, efficient, and scalable solution for cloud storage and file sharing while ensuring data protection and user privacy.

## 1.3 OBJECTIVE

The main objective of the project "Secure Intelligent Cloud Storage with Cryptographic Protection" is to develop a secure and reliable system for storing and sharing data in cloud environments. The system aims to ensure data confidentiality, integrity, and secure access by implementing advanced security mechanisms. It focuses on providing secure cloud storage using encryption techniques to protect sensitive data from unauthorized access. The project also implements cryptographic methods such as AES encryption and SHA-256 hashing to ensure data confidentiality and integrity. In addition, a secure authentication system using Multi-Factor Authentication (MFA) with OTP verification is developed to enhance user security. The system incorporates intelligent monitoring using Artificial Intelligence to detect suspicious user activities and potential security threats. Furthermore, it is designed as a web-based application that allows users to securely upload,

store, and share files. A database management system is also used to efficiently manage user information, encrypted data, and activity logs.

A key objective of this project is to guarantee data confidentiality by employing strong encryption mechanisms. Advanced cryptographic algorithms such as Advanced Encryption Standard (AES) are used to encrypt user data before it is stored in the cloud, ensuring that even if unauthorized access occurs, the data remains unreadable. In addition to confidentiality, the system ensures data integrity through the implementation of SHA-256 hashing, which helps detect any unauthorized modification or tampering of stored data.

Another major objective is to implement a secure authentication mechanism that enhances user identity verification. The system integrates Multi-Factor Authentication (MFA), combining traditional username-password login with One-Time Password (OTP) verification. This additional layer of security significantly reduces the risk of unauthorized account access and strengthens overall system protection.

The project also aims to incorporate intelligent security monitoring using concepts from Artificial Intelligence (AI). By analyzing user behavior patterns and system activities, the system can identify anomalies, suspicious login attempts, or unusual data access patterns. This proactive threat detection mechanism enables early identification of potential cyber threats and helps in preventing security breaches.

# CHAPTER 2

# LITERATURE SURVEY

## 2.1 A TWO-FACTOR AUTHENTICATION MECHANISM USING A NOVEL OTP GENERATION ALGORITHM FOR CLOUD APPLICATIONS

R. Konwar, D. Jha focuses on analyzing existing research related to authentication mechanisms and security in cloud applications. Many researchers have proposed One-Time Password (OTP) based authentication systems to enhance user security. OTP is a dynamic password that is valid for a single login session, reducing the risk of password theft and unauthorized access.

However, traditional OTP systems that rely on SMS or email delivery often suffer from delays and are vulnerable to interception attacks. To overcome these issues, Time-Based One-Time Password (TOTP) algorithms were introduced, where the OTP is generated based on the current time and a shared secret key, improving security but introducing challenges such as time synchronization and device dependency.

Furthermore, Multi-Factor Authentication (MFA) systems combine multiple verification methods, such as passwords and OTPs, to provide stronger security in cloud environments, though they may increase system complexity and user effort. Some advanced systems also integrate biometric authentication with OTP to achieve higher levels of protection, but they require additional hardware and raise privacy concerns. Despite these advancements, existing systems still face issues such as predictability of OTPs, vulnerability to phishing and man-in-the-middle attacks, and inefficiency in cloud-based environments.

## 2.2 EFFICIENT CHAOS BASED HOMOMORPHIC PERMUTATION AND SPM-BOW ALGORITHM FOR SECURE IMAGE STORAGE AND RETRIEVAL IN CLOUD

Anitha Ramachandran and Iyswarya R present a literature survey examines existing research on secure image storage and retrieval in cloud environments. With the rapid growth of cloud computing, storing and accessing images securely has become a major concern due to privacy and data leakage issues. Many researchers have proposed encryption-based techniques to protect image data, among which homomorphic encryption plays an important role as it allows computations to be performed directly on encrypted data without decryption.

In addition, chaos-based encryption methods are widely used because of their high sensitivity to initial conditions and strong randomness, making them suitable for secure image protection. Some studies also focus on permutation techniques to shuffle image pixels, thereby increasing security against attacks. For efficient image retrieval, algorithms like Bag of Words (BOW) and Spatial Pyramid Matching (SPM) are used to extract and match image features effectively.

The combination of SPM and BOW improves accuracy in retrieving similar images from large datasets. However, existing methods often suffer from high computational complexity, storage overhead, and slower retrieval performance in large-scale cloud systems. Furthermore, balancing security and efficiency remains a challenging task. To address these limitations, this project proposes an efficient chaos-based homomorphic permutation technique along with the SPM-BOW algorithm to ensure secure storage and fast retrieval of images in cloud environments. This approach aims to enhance data security, reduce processing time, and improve retrieval accuracy, making it suitable for modern cloud-based applications.

## 2.3 PRIVACY PRESERVING FILE SHARING SYSTEM

M. Lee, K. Chen introduces research related to privacy-preserving file sharing systems in cloud environments. With the increasing use of cloud storage, secure file sharing has become a critical concern due to risks such as unauthorized access, data and privacy breaches. Many researchers have proposed encryption-based approaches, where files are encrypted before being uploaded to the cloud, ensuring that authorized users can access the data.

Techniques such as symmetric and asymmetric encryption, attribute-based encryption (ABE), and secure key management systems are widely used to protect file confidentiality. In addition, access control mechanisms are implemented to restrict file sharing based on user roles and permissions. Some systems also incorporate digital signatures and hash functions to ensure data integrity and authenticity. However, existing solutions often face challenges such as complex key management, increased computational overhead, and difficulty in handling dynamic user access.

Furthermore, ensuring both strong privacy and efficient file sharing remains a major issue in large-scale cloud systems. To address these limitations, this project proposes a privacy-preserving file sharing system that enhances data security while maintaining efficient access and sharing capabilities. The system aims to provide secure encryption, flexible access control, and improved performance, making it suitable for modern cloud-based applications.

## 2.4 SECURE FILE SHARING IN CLOUD

J. Smith, A. Kumar present a existing research related to secure file sharing in cloud computing environments. With the rapid adoption of cloud services, sharing files over the cloud has become common, but it also introduces serious security challenges such as data breaches, unauthorized access, and loss of confidentiality.

To address these issues, many researchers have proposed encryption-based techniques where files are encrypted before being uploaded to the cloud, ensuring that only authorized users can decrypt and access the data. Both symmetric and asymmetric encryption methods are widely used, along with advanced techniques such as attribute-based encryption (ABE) for fine-grained access control.

Additionally, access control mechanisms and authentication techniques are implemented to verify user identity and restrict file access based on roles and permissions. Some systems also use digital signatures and hashing algorithms to ensure data integrity and authenticity during file sharing. However, existing solutions face challenges such as complex key management, increased computational overhead, and scalability issues in large cloud environments.

Furthermore, maintaining a balance between strong security and system performance remains a significant concern. To overcome these limitations, this project proposes a secure file sharing mechanism in the cloud that enhances data protection, simplifies access control, and improves efficiency, making it suitable for modern cloud-based applications.

## 2.5 SECURE CLOUD FILE SHARING USING AES ENCRYPTION

R. Konda, S. Patel focuses on existing research related to secure file sharing in cloud environments using encryption techniques such as AES. With the rapid growth of cloud computing, ensuring the security and privacy of shared files has become a major concern due to threats like unauthorized access, data leakage, and cyber-attacks. Many researchers have proposed encryption-based solutions, where Advanced Encryption Standard (AES) is widely used due to its high security, efficiency, and speed in protecting data.

In addition to encryption, artificial intelligence (AI) techniques are increasingly being integrated into cloud systems to enhance security by detecting anomalies, predicting threats, and managing access control dynamically. Machine learning algorithms can analyze user behaviour patterns and identify suspicious activities in real time, thereby improving the overall security of file sharing systems.

However, existing methods face challenges such as high computational overhead, complexity in integrating AI models, and difficulties in maintaining scalability in large cloud environments. Furthermore, balancing strong encryption with system performance remains a key issue. To address these limitations, this project proposes a Secure Intelligent Cloud Storage with Cryptographic Protection using AES encryption, which aims to provide robust data protection, intelligent threat detection, and efficient file sharing, making it suitable for modern cloud-based applications.

## 2.6 SECURE DATA SHARING IN CLOUD COMPUTING

K. Zhang, X. Wang present a research related to secure data sharing in cloud computing. with the increasing use of cloud platforms for storing and sharing data, ensuring secure and controlled data access has become a major concern. users often need to share sensitive information over the cloud, which exposes data to risks such as unauthorized access, data leakage, and privacy violations. traditional data sharing methods lack strong security mechanisms, making them vulnerable to various cyber threats.

To address these challenges, researchers have proposed advanced cryptographic techniques for secure data sharing. Methods such as encryption, attribute-based encryption (ABE), and identity-based encryption (IBE) are widely used to protect data confidentiality. These techniques ensure that only authorized users with valid credentials can access the shared data. In addition, access control mechanisms are implemented to define user permissions and restrict unauthorized usage.

Many systems also incorporate secure key management and data integrity verification techniques to enhance trust in cloud environments. Despite these advancements, existing systems face challenges such as high computational overhead, complex key management, and difficulties in handling dynamic user access.

Furthermore, maintaining scalability and efficiency while ensuring strong security remains a significant issue in large-scale cloud systems. Therefore, this project proposes a secure data sharing mechanism in cloud computing that aims to provide strong data protection, controlled access, and efficient sharing of information while maintaining system performance, scalability, and user convenience.

## 2.7 PRIVACY-PRESERVING DATA SHARING IN CLOUD

L. Liu, J. Li focuses on existing research related to privacy-preserving data sharing in cloud computing environments. With the rapid growth of cloud services, sharing data securely while maintaining user privacy has become a significant challenge due to risks such as data leakage, unauthorized access, and misuse of sensitive information. Many researchers have proposed encryption-based techniques, where data is encrypted before being stored or shared in the cloud, ensuring that only authorized users can access the original information.

Advanced methods such as attribute-based encryption (ABE) and homomorphic encryption are also used to provide fine-grained access control and allow computations on encrypted data without revealing its content. In addition, secure access control mechanisms and authentication techniques are implemented to verify user identity and manage permissions effectively. Some systems also use anonymization and data masking techniques to further protect user privacy. However, existing solutions face challenges such as complex key management, high computational overhead, and reduced system efficiency in large-scale cloud environments. Furthermore, achieving a balance between strong privacy protection and efficient data sharing remains a major concern.

To address these limitations, this project proposes a privacy-preserving data sharing approach in cloud computing that enhances data confidentiality, ensures secure access control, and improves overall performance, making it suitable for modern cloud-based applications.

## 2.8 BLOCKCHAIN BASED SECURE FILE SHARING SYSTEM

A. Sharma, P. Gupta present a existing research related to blockchain-based secure file sharing systems in cloud environments. With the increasing use of cloud storage, ensuring secure and trustworthy file sharing has become a major challenge due to issues such as data tampering, unauthorized access, and lack of transparency. Traditional centralized systems rely on third-party servers, which may lead to single points of failure and security vulnerabilities. To overcome these limitations, researchers have introduced blockchain technology, which provides a decentralized and immutable ledger for secure data management.

In blockchain-based systems, file transactions are recorded in blocks and linked using cryptographic techniques, ensuring data integrity and transparency. Smart contracts are also used to automate access control and enforce sharing policies without human intervention. Additionally, encryption techniques are applied to protect file content before storing it in the cloud, ensuring confidentiality.

However, existing approaches face challenges such as high computational overhead, scalability issues, and increased storage requirements due to blockchain maintenance. Furthermore, integrating blockchain with cloud systems efficiently remains a complex task. To address these limitations, this project proposes a blockchain-based secure file sharing system that enhances data security, ensures transparency, and provides efficient and reliable file sharing in cloud environments.

## 2.9 SECURE DATA STORAGE IN CLOUD

R. Kumar, S. Singh focuses on existing research related to secure data storage in cloud computing. With the rapid growth of cloud services, organizations and individuals increasingly rely on cloud platforms to store sensitive data. However, this widespread adoption has raised significant concerns regarding data security, privacy, and unauthorized access. Traditional cloud storage systems often depend on standard encryption and access control mechanisms, which may not be sufficient to handle advanced cyber threats such as data breaches, insider attacks, and malicious activities.

To overcome these challenges, researchers have introduced artificial intelligence techniques to enhance cloud data security. AI-based systems use machine learning algorithms to monitor user behavior, detect anomalies, and identify potential security threats in real time. Additionally, encryption techniques combined with AI help in securing stored data, while access control models ensure that only authorized users can retrieve sensitive information.

Many modern approaches also integrate deep learning and predictive analytics to improve threat detection and data protection mechanisms. These systems can learn from past data patterns and continuously improve their performance over time.

Furthermore, ensuring scalability and maintaining performance in large-scale cloud environments remain critical concerns. Therefore, this project proposes an AI-based secure data storage system in the cloud that aims to provide enhanced data protection, intelligent threat detection, and efficient data management while ensuring scalability, reliability, and user-friendly access in cloud environments.

## 2.10 MULTI-FACTOR AUTHENTICATION FOR SECURE CLOUD SYSTEMS

D. Patel, M. Shah present existing research related to multi-factor authentication (MFA) for secure cloud systems. With the rapid adoption of cloud computing, ensuring secure access to cloud services has become a critical challenge due to threats such as unauthorized access, identity theft, and data breaches. Traditional authentication methods that rely only on usernames and passwords are no longer sufficient to provide strong security.

To address this issue, researchers have introduced multi-factor authentication techniques that combine multiple verification factors, such as something the user knows (password), something the user has (OTP or security token), and something the user is (biometric data like fingerprint or face recognition). These methods significantly enhance security by adding additional layers of protection. Many systems also integrate OTP-based authentication and mobile-based verification to strengthen user identity validation. However, existing MFA systems face challenges such as increased complexity, higher implementation cost, and potential usability issues for users.

In addition, managing multiple authentication factors efficiently in large-scale cloud environments remains a concern. Therefore, this project proposes a multi-factor authentication mechanism for secure cloud systems that aims to provide strong security, improved user verification, and efficient access control while maintaining usability and performance in cloud environments.

# CHAPTER 3

# EXISTING SYSTEM

In the existing system, users store and share files using traditional cloud storage platforms such as public cloud services and file-sharing websites. These platforms allow users to upload, download, and share files easily over the internet. While they provide convenience and accessibility, most of these systems focus primarily on usability rather than strong security measures. As a result, users often face risks related to data privacy and protection.

One of the major limitations of the existing system is the lack of strong encryption mechanisms. In many cloud storage platforms, data is either stored without proper encryption or relies on basic security techniques. This makes sensitive information vulnerable to unauthorized access and cyber-attacks. If attackers gain access to the system, they can easily view or misuse confidential data.

Another drawback is the use of weak authentication methods. Most systems rely only on username and password for user verification. Since passwords can be easily guessed, stolen, or hacked, unauthorized users may gain access to accounts. The absence of multi-factor authentication increases the risk of identity theft and data breaches.

In addition, existing systems often do not provide proper data integrity verification. Users have no guarantee that the files stored in the cloud remain unchanged. During transmission or storage, files may be altered either intentionally or unintentionally, and users may not be able to detect such modifications. This creates serious concerns regarding the reliability and authenticity of data.

Another limitation is the lack of activity monitoring and access control mechanisms. Many cloud platforms do not track user actions in detail, making it difficult to detect suspicious activities. Without proper monitoring, unauthorized file access or sharing may go unnoticed, leading to security issues.

Furthermore, existing systems do not provide a fully secure file-sharing environment. Files shared between users may not be properly protected, increasing the chances of interception or misuse during transmission. This reduces user trust in cloud storage systems, especially when dealing with sensitive or confidential information.

Due to these limitations, traditional cloud storage and file-sharing systems fail to provide complete security in terms of confidentiality, integrity, and authentication. Therefore, there is a need for an advanced system that integrates strong encryption, secure authentication, data integrity verification, and activity monitoring to ensure safe and reliable cloud file sharing.

## 3.1 EXISTING SYSTEM ARCHITECTURE

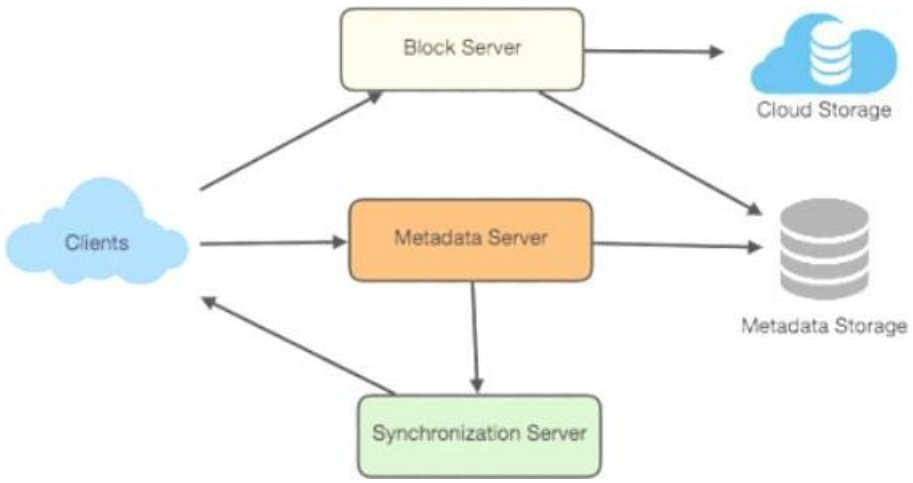


**Figure 3.1 Existing System Architecture**

This architecture represents the workflow of the existing cloud file storage and sharing system. In the current process, a user first logs into a cloud platform using basic authentication such as username and password. After logging in, the user uploads files directly to the cloud server, where the data may be stored without strong encryption.

# CHAPTER 4

# PROBLEM IDENTIFIED

In the current digital environment, cloud storage systems are widely used by individuals and organizations to store, access, and share data. Although these systems provide convenience and accessibility, they often fail to ensure adequate security for sensitive information. Users rely on various cloud platforms to upload and share files, but many of these platforms do not provide strong protection mechanisms against cyber threats. As a result, users face significant challenges in maintaining data confidentiality, integrity, and secure access.

One of the major problems in the existing system is the risk of unauthorized access. Most cloud storage platforms use basic authentication methods such as username and password. These methods are not sufficiently secure, as passwords can be easily guessed, stolen, or compromised through phishing or hacking attacks. Without additional layers of security, unauthorized users may gain access to sensitive files, leading to serious privacy and security issues.

Another important issue is the lack of strong data encryption. In many traditional systems, files are stored in the cloud without proper encryption or with weak encryption techniques. This makes the data vulnerable to cyber-attacks, as attackers can access and read the stored information if they breach the system. Without effective encryption, users cannot ensure the confidentiality of their data, especially when dealing with personal or confidential files. The absence of data integrity verification is another significant problem. Existing systems do not always provide mechanisms to verify whether the data has been altered during storage or transmission.

Another limitation is the lack of multi-factor authentication mechanisms. Most systems rely on single-factor authentication, which increases the risk of unauthorized access. Without additional verification methods such as One-Time Password (OTP), it becomes difficult to ensure that the person accessing the system is the legitimate user. This weakness can lead to identity theft and misuse of sensitive information.

In addition, existing systems often do not provide proper monitoring and tracking of user activities. There is limited visibility into who accessed the data, when it was accessed, and what actions were performed. This lack of activity monitoring makes it difficult to detect suspicious behavior or prevent unauthorized operations within the system.

Another challenge is the insecure file-sharing process. Files shared between users may not be adequately protected during transmission, increasing the chances of interception or misuse. In many cases, file-sharing links can be accessed by unintended users if not properly secured, leading to potential data leaks.

Furthermore, existing systems do not provide an integrated security approach. Security features such as encryption, authentication, and integrity verification are often implemented separately or not fully utilized. This results in incomplete protection and increases the chances of vulnerabilities in the system.

Due to these limitations, current cloud storage and file-sharing systems do not provide a fully secure environment for managing sensitive data. The lack of strong authentication, encryption, integrity verification, and monitoring creates significant risks for users. Therefore, there is a need for an advanced system that integrates multiple security mechanisms, including AES encryption, SHA-256 integrity verification, and Multi-Factor Authentication, to ensure safe and reliable cloud file sharing.

# CHAPTER 5

# PROPOSED SYSTEM

The proposed system is a Secure Intelligent Cloud Storage with Cryptographic Protection (GhostKey) designed to provide a safe and reliable environment for storing and sharing files in the cloud. This system focuses on enhancing data security by integrating advanced techniques such as encryption, authentication, and integrity verification. It ensures that only authorized users can access and manage files, thereby reducing the risk of data breaches and unauthorized access.

The system provides a centralized platform where users can securely upload, store, and share files. Before storing the files in the cloud, the system encrypts them using AES (Advanced Encryption Standard). This ensures that the data remains confidential and cannot be accessed by unauthorized users even if the storage is compromised.

Another important feature of the proposed system is secure user authentication. The system uses Multi-Factor Authentication (MFA) with dynamic OTP (One-Time Password) verification. This ensures that only verified users can log in and perform operations such as uploading or sharing files. This additional layer of security prevents unauthorized access and enhances user trust.

The system also includes a data integrity verification mechanism using SHA-256 hashing. Each file is assigned a unique hash value, which is used to verify that the file has not been altered during storage or transmission. This helps maintain the reliability and authenticity of the data.

In addition, the system supports secure file sharing between users. Files can be shared only with authorized users, and access is controlled to prevent misuse. The system also monitors user activities, providing transparency and helping detect any suspicious actions.

Overall, the proposed system provides a complete security solution by combining encryption, authentication, and monitoring features, ensuring safe and efficient cloud file management.

## 5.1 PROPOSED SYSTEM ARCHITECTURE

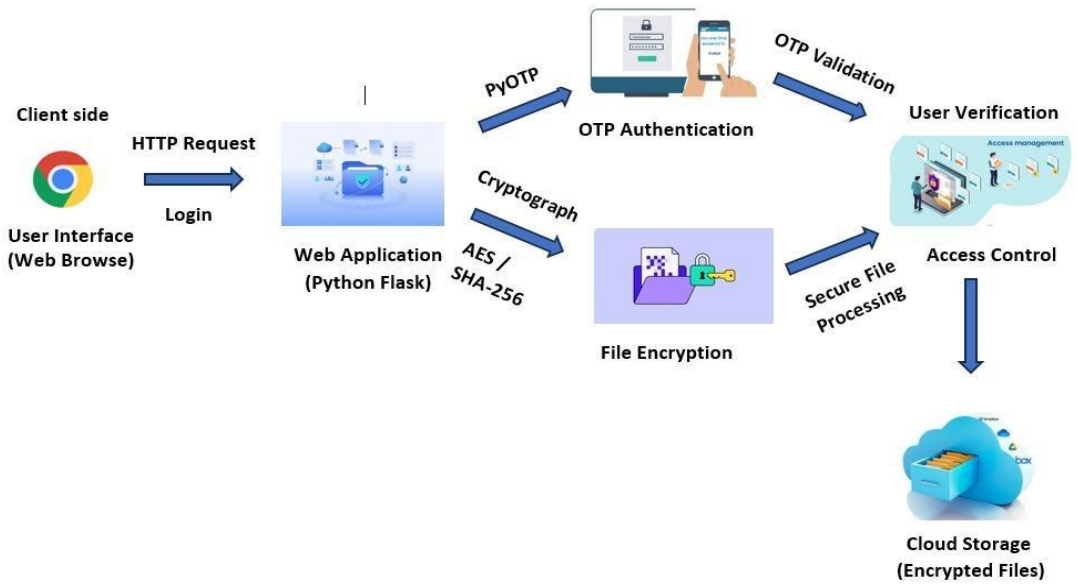


**Fig.5.1. Proposed System Architecture**

The architecture of the proposed system consists of multiple modules that work together to ensure secure file storage and sharing.

The process begins with User Authentication, where the user registers and logs into the system using credentials and OTP verification. This step ensures that only authorized users can access the system.

# CHAPTER 6

# SYSTEM REQUIREMENTS

## 6.1 HARDWARE REQUIREMENTS

| | | |
|---|---|---|
| Processor | - | Intel i5 or above / AMD Ryzen |
| RAM | - | Minimum 8GB |
| Storage | - | Minimum 1 GB free |
| GPU | - | NVIDIA GPU with CUDA support |

## 6.2 SOFTWARE REQUIREMENTS

| | | |
|---|---|---|
| Operating System | - | Windows 10 or MacOS |
| Programming Language | - | Python |
| Security Technologies | - | AES Encryption, SHA-256 Hashing, Multi-Factor Authentication (OTP) |
| Front End Development | - | HTML, CSS, JavaScript |
| Database | - | MySQL / SQLite |

# CHAPTER 7

# SYSTEM IMPLEMENTATION

## 7.1 MODULES

1. User Authentication Module
2. OTP Generation and Verification Module
3. File Encryption and Upload Module
4. Secure Cloud Storage Module
5. File Sharing and Activity Monitoring Module

### 7.1.1 User Authentication Module

The User Authentication module is the initial stage of the proposed system and plays a crucial role in ensuring secure access. This module allows users to register by providing basic details such as username, email ID, and password. The registration process creates a unique account for each user, ensuring that their data and activities are stored securely.

After successful registration, users can log in to the system using their credentials. The system verifies the entered username and password to authenticate the user's identity. This process helps prevent unauthorized access and ensures that only valid users can access the system features.

Once the user logs in successfully, they can access the dashboard and perform operations such as uploading, downloading, and sharing files securely. This module ensures privacy and protects user data from unauthorized access.

### 7.1.2 OTP Generation and Verification Module

The OTP Generation and Verification module adds an extra layer of security to the system through Multi-Factor Authentication. After entering valid login credentials, the system generates a One-Time Password (OTP) and sends it to the user's registered email.

The user must enter the correct OTP to complete the login process. This ensures that even if login credentials are compromised, unauthorized users cannot access the system without OTP verification.

This module enhances system security by providing dynamic authentication and preventing identity theft or unauthorized login attempts. It plays a vital role in protecting sensitive data stored in the system.

### 7.1.3 File Encryption and Upload Module

The File Encryption and Upload module is responsible for securing files before storing them in the cloud. In this module, users can upload files such as documents, images, or other data.

Before storing the file, the system encrypts it using AES (Advanced Encryption Standard). Encryption converts the file into an unreadable format, ensuring that only authorized users can access its content.

This process guarantees data confidentiality, as even if the storage is compromised, the encrypted files cannot be accessed without proper decryption. The encrypted files are then securely stored in the cloud storage system.

### 7.1.4 Secure Cloud Storage Module

The Secure Cloud Storage module manages the storage of encrypted files. All files uploaded by users are stored in encrypted form, ensuring that sensitive data is protected at all times.

In addition to storage, this module uses SHA-256 hashing to generate a unique hash value for each file. This hash value is used to verify data integrity and detect any unauthorized modifications.

When a file is retrieved, the system compares the current hash value with the original hash to ensure that the file has not been altered. This guarantees that the data remains accurate and trustworthy.

### 7.1.5 File Sharing and Activity Monitoring Module

The File Sharing and Activity Monitoring module allows users to securely share files with other authorized users. Access control mechanisms ensure that only permitted users can view or download shared files.

The system also tracks user activities such as login, file upload, download, and sharing. This monitoring helps in detecting suspicious behaviour and maintaining system transparency. By maintaining logs of user actions, the system ensures accountability and improves overall security. This module plays an important role in providing a safe and controlled file-sharing environment.

# CHAPTER 8

# SYSTEM TESTING

## 8.1 Unit Testing

Unit Testing is performed to verify the correctness of individual components of the system. Each module is tested independently to ensure that it functions as expected without errors. This helps in identifying bugs at an early stage before integrating different modules.

In the GhostKey system, unit testing was carried out on various backend modules developed using the Flask framework. Individual functions such as user authentication, OTP generation, file encryption, file decryption, and hashing were tested separately.

For example, the AES encryption module was tested by uploading sample files and verifying whether the encrypted output was correctly generated and could be decrypted back to the original file. Similarly, the SHA-256 hashing module was tested to ensure that unique hash values were generated for each file and that any modification in the file resulted in a different hash value.

The OTP generation module was tested by sending OTPs to registered email addresses and verifying their correctness during login. Each API endpoint related to file upload, download, and sharing was also tested using sample inputs.

Unit testing ensured that each component functioned correctly, improving the reliability and stability of the system.

## 8.2 Integration Testing

Integration Testing is performed to ensure that different modules of the system work together correctly after integration. The main objective is to verify proper communication between components.

In this project, integration testing was carried out between the frontend interface and backend services developed using Flask. The frontend collects user inputs such as login details, file uploads, and sharing requests, while the backend processes these requests and returns appropriate responses. During testing, various workflows were verified. For instance, when a user logs in, the authentication module validates the credentials and triggers OTP verification. After successful authentication, the user uploads a file, which is then passed to the encryption module before being stored in the cloud.

Similarly, when a file is requested for download, the system retrieves the encrypted file, verifies its integrity using SHA-256 hashing, and then decrypts it before sending it to the user.

Integration testing confirmed that all modules interact correctly and that data flows smoothly across the system.

## 8.3 System Testing

System Testing evaluates the complete and integrated system to ensure that it meets all functional and non-functional requirements. The entire application is tested as a whole in a real-world environment.

In the GhostKey system, system testing involved verifying all functionalities such as user registration, login with OTP, file upload,

encryption, storage, file sharing, and download. Different user scenarios were tested to ensure that the system operates smoothly under normal conditions.

The application was deployed in a web environment and accessed through browsers to simulate real usage. The system was tested to ensure that files were securely stored, properly encrypted, and accessible only to authorized users.

System testing confirmed that the application meets all security requirements and performs reliably in real-world conditions.

## 8.4 Performance Testing

Performance Testing is conducted to evaluate the system's performance under different conditions such as multiple users, large file sizes, and continuous usage. The goal is to ensure that the system maintains acceptable response time and stability.

In this project, performance testing focused on measuring the time taken for file encryption, decryption, upload, and download operations. The system was tested with different file sizes to ensure efficient processing without delays.

The OTP authentication process was also tested to verify quick delivery and validation. Additionally, the system was tested with multiple user requests to ensure that the server could handle concurrent operations without failure.

The results showed that the system performs efficiently and maintains stability even under heavy usage conditions.

## 8.5 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) is performed to ensure that the system meets user expectations and fulfills its intended purpose. In this stage, real users interact with the system and provide feedback on usability and functionality.

For this project, users were asked to test the GhostKey system by registering, logging in with OTP, uploading files, and sharing them securely. They evaluated the system based on ease of use, security features, and overall performance.

Users found the system easy to use and appreciated the added security features such as encryption and OTP-based authentication. The secure file-sharing process and activity monitoring features were also well received.

Based on user feedback, minor improvements were made to enhance user interface design and system responsiveness. Overall, the system was found to be reliable, secure, and effective for cloud file sharing.

# CHAPTER 9

# RESULT AND DISCUSSION

The proposed Secure Intelligent Cloud Storage with Cryptographic Protection (GhostKey) was successfully designed and developed to provide a secure platform for storing and sharing files. The system integrates advanced security mechanisms such as encryption, authentication, and data integrity verification to ensure the protection of user data. Through this system, users can securely upload, store, and share files while maintaining confidentiality and reliability.

Initially, the User Authentication module allows users to register and log in securely to the system. This module ensures that only authorized users can access the platform and perform operations. The implementation of authentication mechanisms helps protect user accounts and prevents unauthorized access to sensitive data.

The OTP Generation and Verification module enhances security by providing Multi-Factor Authentication. During login, users receive a One-Time Password (OTP) to verify their identity. This additional layer of security ensures that even if login credentials are compromised, unauthorized users cannot access the system. The results show that OTP-based authentication significantly improves system security and user trust.

The File Encryption and Upload module ensures that all files are encrypted before being stored in the cloud. AES encryption converts files into a secure format, making them unreadable to unauthorized users. The testing results confirm that encrypted files can only be accessed after proper decryption, ensuring complete confidentiality of user data.

The Secure Cloud Storage module stores encrypted files and maintains their integrity using SHA-256 hashing. Each file is assigned a unique hash value, which is used to verify whether the file has been altered. The results demonstrate that any modification in the file results in a change in the hash value, thereby ensuring data integrity and reliability.

The File Sharing and Activity Monitoring module enables users to share files securely with authorized users. Access control mechanisms ensure that only permitted users can view or download shared files. Additionally, the system records user activities such as login, upload, download, and sharing. This monitoring helps detect suspicious actions and improves overall system transparency.

The overall results of the system indicate that it effectively addresses the security challenges present in traditional cloud storage systems. Users can securely manage their files without worrying about unauthorized access or data tampering. The integration of encryption, authentication, and monitoring provides a comprehensive security solution.

From the discussion, it is clear that the proposed system improves data protection, enhances user trust, and provides a reliable cloud file-sharing environment. The system successfully meets the objectives of ensuring confidentiality, integrity, and secure access, making it suitable for real-world applications.

# CHAPTER 10

# CONCLUSION AND FUTUREWORK

## 10.1 CONCLUSION

The Secure Intelligent Cloud Storage with Cryptographic Protection (GhostKey) was developed to provide a secure and reliable platform for storing and sharing files in cloud environments. The system focuses on addressing major security challenges such as unauthorized access, data breaches, and data tampering by integrating advanced security mechanisms.

The system combines multiple modules such as User Authentication, OTP Verification, File Encryption, Secure Cloud Storage, and File Sharing with Activity Monitoring to provide a complete security solution. The User Authentication and OTP modules ensure that only authorized users can access the system, thereby protecting user accounts and sensitive data.

The File Encryption module uses AES encryption to secure files before storing them in the cloud, ensuring data confidentiality. The Secure Cloud Storage module, along with SHA-256 hashing, maintains data integrity by detecting any unauthorized modifications. These features ensure that the stored data remains safe, accurate, and reliable.

The File Sharing and Activity Monitoring module enables users to share files securely while maintaining control over access permissions. It also tracks user activities, providing transparency and helping to detect suspicious behaviour. This enhances overall system security and accountability.

Overall, the proposed system successfully achieves its objectives of providing confidentiality, integrity, and secure access to cloud-stored data. It offers a user-friendly interface along with strong security features, making it suitable for real-world applications. The system improves user trust and ensures safe file management in cloud environments.

## 10.2 FUTURE WORK

The proposed system can be further enhanced by incorporating additional advanced features to improve security and usability. In the future, the system can be integrated with real-time cloud services and distributed storage systems to provide better scalability and performance. This will allow users to store and access large amounts of data more efficiently.

Another improvement can be the integration of advanced Artificial Intelligence and Machine Learning techniques for detecting unusual user behaviour and potential security threats. These technologies can help identify suspicious activities such as unauthorized access attempts and provide real-time alerts to users.

The system can also be enhanced by implementing biometric authentication methods such as fingerprint or facial recognition, which will further strengthen user verification and security. In addition, advanced access control mechanisms such as role-based permissions can be introduced to provide more flexibility in file sharing.

Future versions of the system can include features such as secure file expiration, automated backup, and recovery mechanisms to prevent data loss. Mobile application support can also be added so that users can securely access and manage their files from anywhere at any time.

Furthermore, the system can be extended to support end-to-end encryption and blockchain-based data verification to provide even higher levels of security and transparency. By implementing these improvements, the system can evolve into a more powerful and comprehensive secure cloud file-sharing platform.

# APPENDIX – A

# SOURCE CODE

```
import warnings

warnings.filterwarnings("ignore")

from flask import Flask, render_template, request, redirect, session, send_file,
jsonify

import os, time, pyotp, sqlite3, io, hashlib

from encryption import encrypt_file, decrypt_file

from otp_utils import generate_otp, verify_otp

from email_utils import send_email_otp

from sms_utils import send_sms_otp

app = Flask(__name__)

app.secret_key = "ghostkey_secure"

UPLOAD_FOLDER = "uploads"

os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# ------------------ MEMORY STORAGE ------------------

login_attempts = {}

secure_links = {}

security_logs = []

intrusion_logs = []

# ------------------ LOG FUNCTIONS ------------------

def add_log(user, action):

security_logs.append({"user": user, "action": action})

def add_intrusion(user, alert):
```

```
intrusion_logs.append({"user": user, "alert": alert})
# ------------------ DATABASE ------------------
def init_db():
conn = sqlite3.connect("ghostkey.db")
c = conn.cursor()
c.execute("""
CREATE TABLE IF NOT EXISTS users(
email TEXT PRIMARY KEY,
password TEXT,
phone TEXT
)
""")
c.execute("""
CREATE TABLE IF NOT EXISTS file_keys(
user TEXT,
filename TEXT,
encryption_key TEXT,
file_hash TEXT
)
""")
conn.commit()
conn.close()
init_db()
# ------------------ HOME ------------------
@app.route("/")
```

```
def index():
return render_template("index.html")
# ------------------ REGISTER ------------------
@app.route("/register", methods=["GET","POST"])
def register():
if request.method == "POST":
email = request.form["email"]
password = request.form["password"]
phone = request.form["phone"]
password_hash = hashlib.sha256(password.encode()).hexdigest()
conn = sqlite3.connect("ghostkey.db")
c = conn.cursor()
try:
c.execute(
"INSERT INTO users VALUES (?, ?, ?)",
(email, password_hash, phone)
)
conn.commit()
except:
conn.close()
return "User already exists"
conn.close()
add_log(email,"User registered")
return redirect("/login")
return render_template("register.html")
```

```
# ------------------ LOGIN ------------------
@app.route("/login", methods=["GET","POST"])
def login():
if request.method == "POST":
email = request.form["email"]
password = request.form["password"]
password_hash = hashlib.sha256(password.encode()).hexdigest()
conn = sqlite3.connect("ghostkey.db")
c = conn.cursor()
c.execute("SELECT password, phone FROM users WHERE email=?",
(email,))
result = c.fetchone()
conn.close()
if result and result[0] == password_hash:
phone = result[1]
secret = pyotp.random_base32()
otp = generate_otp(secret)
session["email"] = email
session["otp_secret"] = secret
send_email_otp(email, otp)
send_sms_otp(phone, otp)
print("LOGIN OTP:", otp)
add_log(email,"Login OTP sent")
return render_template("otp.html")
else:
```

```
add_intrusion(email,"Invalid login attempt")
return "Invalid email or password"
return render_template("login.html")
# ------------------ VERIFY OTP ------------------
@app.route("/verify", methods=["POST"])
def verify():
otp = request.form["otp"]
user = session.get("email")
if not user:
return redirect("/login")
if verify_otp(session["otp_secret"], otp):
login_attempts[user] = 0
add_log(user,"Login success")
return redirect("/dashboard")
else:
login_attempts[user] = login_attempts.get(user,0)+1
if login_attempts[user] >= 3:
add_intrusion(user,"Multiple OTP failures")
return "Invalid OTP"
# ---------------- GENERATE SHARE LINK ----------------
@app.route("/generate_link/<filename>")
def generate_link(filename):
if "email" not in session:
return redirect("/login")
filepath = os.path.join(UPLOAD_FOLDER, filename)
```

```
if not os.path.exists(filepath):
return "File not found"
token = str(time.time())
expiry = time.time() + 120
secure_links[token] = {
"file": filename,
"expiry": expiry
}
link = f"http://127.0.0.1:5000/secure_download/{token}"
add_log(session["email"], f"Share link generated for {filename}")
return render_template("share_link.html", link=link)
# ---------------- SECURE DOWNLOAD ----------------
@app.route("/secure_download/<token>")
def secure_download(token):
if token not in secure_links:
return "Invalid or expired link"
data = secure_links[token]
if time.time() > data["expiry"]:
return "Download link expired"
filename = data["file"]
path = os.path.join(UPLOAD_FOLDER, filename)
with open(path,"rb") as f:
encrypted = f.read()
add_log("external_user", f"Secure link download: {filename}")
return send_file(
```

```
io.BytesIO(encrypted),
download_name=filename,
as_attachment=True
)
@app.route("/decrypt", methods=["GET","POST"])
def decrypt_page():
if "email" not in session:
return redirect("/login")
if request.method=="POST":
file=request.files["file"]
password=request.form["password"]
encrypted=file.read()
key=hashlib.sha256(password.encode()).digest()
try:
decrypted=decrypt_file(encrypted,key)
except:
return "Wrong password"
return send_file(
io.BytesIO(decrypted),
download_name=file.filename.replace(".enc",""),
as_attachment=True
)
return render_template("decrypt.html")
@app.route("/share", methods=["GET","POST"])
def share_page():
```

```
# ---------------- SECURITY DASHBOARD ----------------
@app.route("/security_dashboard")
def security_dashboard()
if "email" not in session:
return redrect("/login")
return render_template(
"security_dashboard.html",
logs=security_logs,
intrusions=intrusion_logs
)
# ------------------ CHATBOT -----------------
@app.route("/chatbot")
def chatbot():
return render_template("chatbot.html")
@app.route("/ask", methods=["POST"])
def ask()
q = request.form.get("message","").lower(
if "encryption" in q:
ans = "Encryption converts plaintext into ciphertext using AES algorithm."
elif "aes" in q:
ans = "AES is a symmetric encryption algorithm used to secure files."
elif "decryption" in q:
ans = "Decryption converts encrypted ciphertext back into readable plaintext."
else:
ans = "Ask me about encryption, AES, or cybersecurity."
```

```
return jsonify({"response": ans})
# ------------------ LOGOUT ------------------
@app.route("/logout")
def logout():
user = session.get("email","unknown")
add_log(user,"User logged out")
session.clear()
return redirect("/login")
# ------------------ RUN ------------------
if __name___== "__main__":
app.run(debug=True)
```

# APPENDIX – B

# SCREENSHOTS

## Output

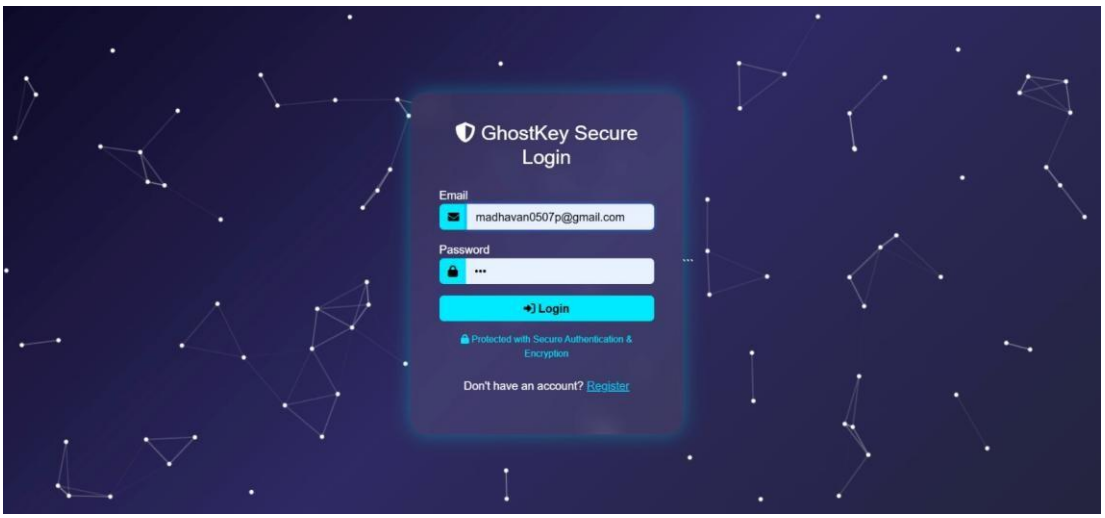


**Figure. B.1 User Registration and Authentication**

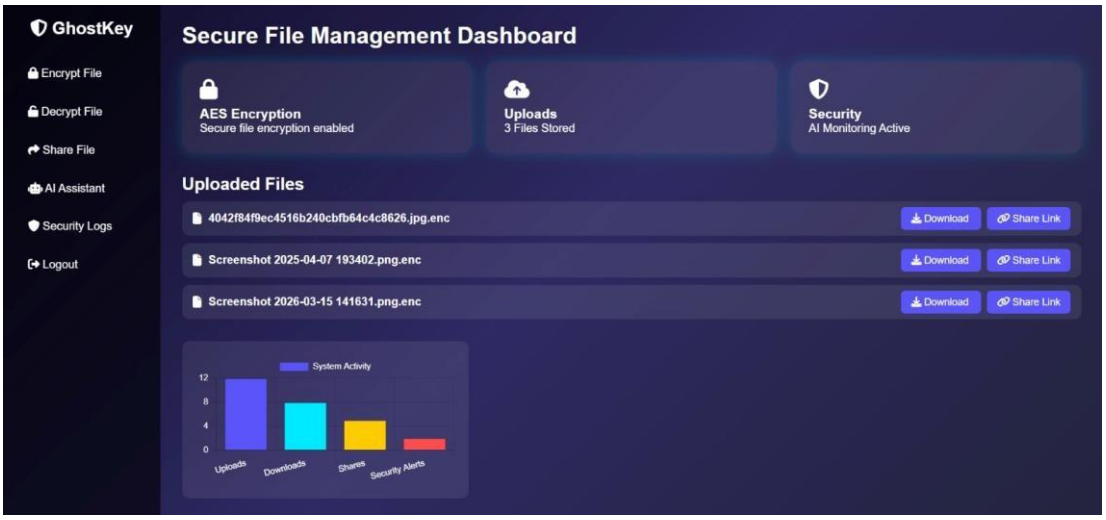


**Figure. B.2 Dashboard Management**

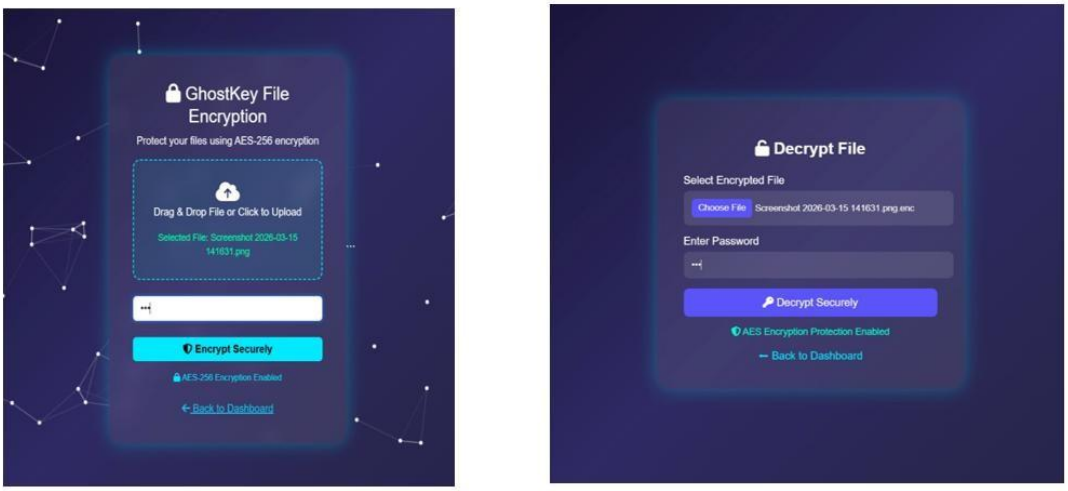


**Figure. B.3 File Encryption and Decryption**

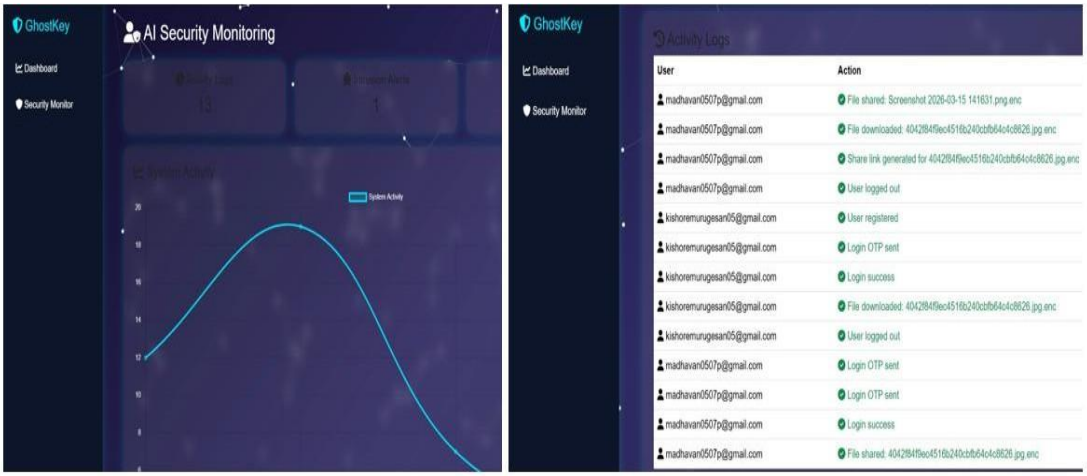


**Figure. B.4 Secure Cloud Storage**

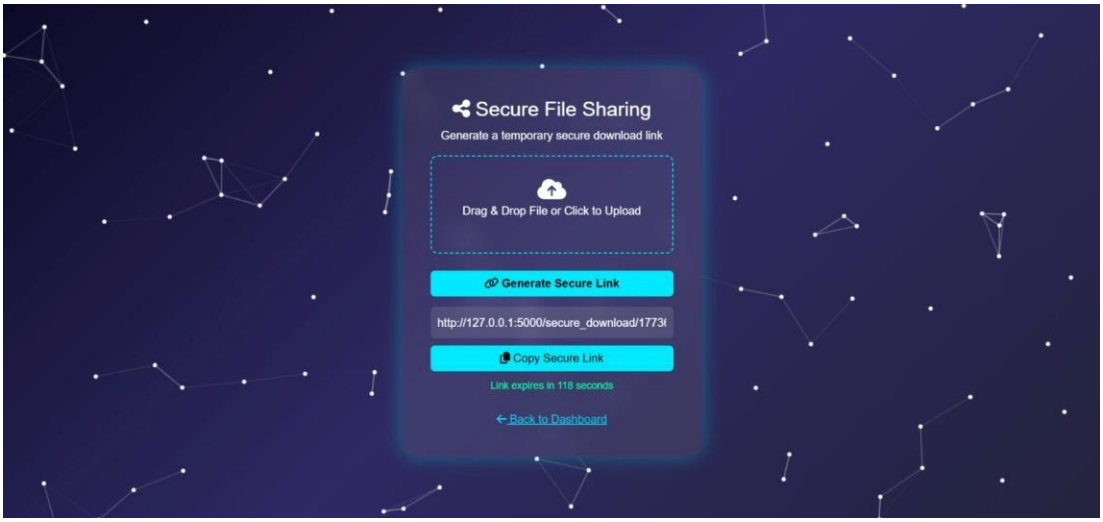


**Figure. B.5 File Sharing and Activity Monitoring**

# REFERENCES


1. Anitha Ramachandran and Iyswarya R, “Efficient Chaos Based Homomorphic Permutation and SPM-BOW Algorithm for Secure Image Storage and Retrieval in Cloud,” International Journal of Advanced Research in Computer Science, 2020.
2. R. Konda and S. Patel, “Secure Cloud File Sharing Using AES Encryption,” International Journal of Information Security and Privacy, 2020.
3. R. Konwar and D. Jha, “A Two-Factor Authentication Mechanism Using a Novel OTP Generation Algorithm for Cloud Applications,” International Journal of Computer Security, 2021.
4. R. Kumar and S. Singh, “Secure Data Storage in Cloud Computing,” International Journal of Cloud Applications and Computing, 2021.
5. M. Lee and K. Chen, “Privacy Preserving File Sharing System,” International Journal of Network Security, 2018.
6. L. Liu and J. Li, “Privacy-Preserving Data Sharing in Cloud Computing,” Journal of Cloud Computing, 2019.
7. D. Patel and M. Shah, “Multi-Factor Authentication for Secure Cloud Systems,” International Journal of Information Security, 2021.
8. A. Sharma and P. Gupta, “Blockchain Based Secure File Sharing System,” International Journal of Cloud Computing and Security, 2020.
9. J. Smith and A. Kumar, “Secure File Sharing in Cloud Computing,” International Journal of Computer Applications, 2019.
10. K. Zhang and X. Wang, “Secure Data Sharing in Cloud Computing,” IEEE International Conference on Cloud Computing, 2018.